Handoff 29: Function-call checks (master run S05, ENG-06)

Date: 2026-11-02 (America/Toronto)
Written by: Claude Code (implementer), in a separate git worktree
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request (#34), not merged, not deployed. **No database migration, no new dependency, no lesson content.** Nothing a learner sees changes until a lesson uses it.

> **How to use this file:** upload it with `docs/plan/browser-checks.md` and handoffs 11 and 22. The pull request diff is the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing ENG-06 for Python Academy: a browser check can now call a
function the learner defined and compare what it returns or prints. I'm
attaching docs/plan/browser-checks.md and this handoff. Look hardest at section
8 of the contract: the equality rule, and whether refusing hidden calls is right.
Check that ordinary cases really are unchanged, that nothing hidden can reach
the browser, and that the server never runs learner code.
```

Run plan details

Field	Value
Stage	S05, ENG-06
Base	`main`, at `4b70a28`. Nothing unmerged is needed. **No other PR needs to merge first**
Part 2 boxes relied on	**None. No box is ticked.** S05's gate is "none"
With #32	A trial merge has no conflict (section 7)

1. Step 0 (checks before building)

Check	Result
`git fetch --all`, `gh pr list --state all`	Done. Open: #32, and #33 from this run

Check	Result
ID check	ENG-06 does not appear in `build.md`
Where the work happened	A separate worktree, `../python-academy-fn`, on `eng/function-checks` from `origin/main`. The main folder was not switched

2. Handoff record (build.md template)

Field	Value
Task	Extend the ENG-03 contract, additively, so a check can call a learner's function with arguments and compare the return value or printed output
Human goal	Unit 5 (functions) can be checked fairly, and "you printed instead of returning" gets a clear message
Repository and branch	`torontojohn/python-academy`, branch `eng/function-checks`
Non-goals	Any lesson; hidden calls (section 4); sets as return values; `backlog.md` and `decision-log.md`
Data and compatibility	No migration. API changes are additive: `runs[].returned`, three new `runs[].status` values, and `call` and `returns` in a plan, only for a case that has a call. An old page with new content gets `service_error`. A new page with an old API is not affected until a lesson uses calls
Security	The worker gets new code, but the worker's Content-Security-Policy, the time limit, the output limit, the network removal and the "unsupported import" state are unchanged. The new Python only loads when a plan has a call
Deployment status	Not requested

Files changed

File	Why
`backend/app/content.py`	`call` and `returns` on a case, and their validation
`backend/app/browser_checks.py`	The plan, the equality rule, the judge and its new results
`frontend/public/python-worker.js`	Calls the function and describes what happened

File	Why
`frontend/public/check-runner.js`	Passes the call on, and returns the value
`frontend/app/adventure.tsx`	The Output panel shows "Returned: …" and the new stop reasons
`docs/plan/browser-checks.md` (section 8), `backend/content/README.md`	The contract
`backend/tests/test_function_checks.py` (new, 70 tests), `backend/tests/fixtures/function_checks/` (new), `frontend/tests/function-checks.spec.ts` (new, 27 tests), `frontend/tests/fixtures/function-checks-api.json` (new, recorded real answers)	Section 6
`docs/run-log.md`, this handoff and its PDF	The run log and the record

3. How it works

The full contract is in `docs/plan/browser-checks.md` section 8. In short:

1. A case says `"call": {"function": "double", "args": [2]}` and `"returns": {"value": 4}`.
2. The worker runs the program, throws away what it printed so far, calls `double(2)`, and reports what the call printed and returned.
3. The API compares the value, and never runs anything.

What the learner did	What they see
Right answer	"Check passed…", and "Returned: 4" in the Output panel
Wrong value	"For double(2), your function should return 4, but it returned 3."
Printed instead of returning	"…but it returned None. It printed something instead: printing shows a value, and return hands it back."
No such function, or the wrong name	"Your program should define a function called double that takes 1 argument."
Wrong number of parameters	The same sentence, then "Yours cannot be called that way for double(2)."
Error in the function	The usual friendly explanation, with the line marked
Endless loop in the function	Stopped at 3 seconds, with the usual explanation

What the learner did	What they see
Returned a set or an object	"…returned a value this check cannot compare (set). Return a number, text, True or False, None, a list, a tuple or a dictionary with text keys."

4. Decisions and why

Decision	Options considered	Reason	Who decided
A hidden call is refused when content loads	Send hidden arguments anyway, as section 5 of the contract does for inputs; encrypt them	S05 says hidden arguments never reach the browser. It also says the function is called in the worker, and the server must never run learner code. All three cannot hold for a hidden call, so the only way to keep them is to have no hidden calls. Encrypting would be pretend: the browser must decrypt them to call the function. **This limits unit 5** (question 1)	Owner's rule; implementer's reading (**assumption 1**)
A separate `call` and `returns`, not more fields on `expected`	A `kind` field; an expected value in `expected`	`returns` being present is what says "compare it", so `null` can mean "should return `None`"	Implementer
Only what the call prints is compared	The whole program's output	A correct program may print or call its own function first; that must not fail it	Implementer
A tuple equals a list	Keep them different	`return a, b` and `return [a, b]` are both correct answers, and JSON has no tuple	Implementer (**assumption 2**)
`4` equals `4.0`, and `True` does not equal `1`	Python's `==`, where `True == 1`; strict types	`4.0` is a right number. Python's `True == 1` is a trap: returning `True` for "double(0.5)" would pass. **Differs from Python** on purpose	Implementer (**assumption 3**)
A fraction needs a tolerance, enforced when content loads	Leave it to the author; a default tolerance	S05 asks for no float traps. A mistake stops the deploy	Implementer
Sets, objects and subclasses are `not_comparable`	Compare by `repr`; run the comparison in the browser	Comparison stays on the server and uses plain values only, so a learner's `__eq__` cannot claim a pass. Units 4 and 11 may need more (question 3)	Implementer
Any compatible signature passes	Require an exact parameter count	`def double(n, times=2)` is correct, so it passes. "Accept every correct program"	Implementer
Failures use the case's rule ID	New rule names	No naming is invented, and a hint can already be tagged with a case	Implementer

Decision	Options considered	Reason	Who decided
The Python helper only loads when a plan has a call	Always load it	Ordinary lessons run exactly the same code as before	Implementer
Base on `main`, not #32	Chain on #32	B-R1 is empty, and nothing here needs #32	Plan section 1.3

5. Bugs found and how they were fixed

Bug or near-miss	How it was found	Fix
The new field changed an old recording. `model_dump()` added `returned: null` to every ordinary report, so unit 2's recorded requests no longer matched	Two unit 2 tests failed during the deliberate-break run, in every break, so they were failing without any break too	`exclude=True` on the field. The old fixtures are unchanged, which is what the "additive" rule demands
My test fixture failed content validation: wrong unit ID and an example with replies	Loading it	Fixed the fixture
My test for a function status on an ordinary case stopped at the fixture's required constructs, so it tested the wrong thing	The test failed	The code in the test now has those constructs
Near-miss: a very deep value in `returned` (`[[[[…`) could have crashed the judge	Written as a test first	`RecursionError` is caught and it is a `service_error`. An integer too long for Python to read is caught the same way
Near-miss: a class named with a non-English letter could have made the API refuse the report (422), because `error_type` only allows plain names	Reading the pattern; written as a test	The worker reports the type as `unknown`

6. How it was tested

Commands and results (each suite alone, macOS, Python 3.12.14, Node 24)

Command	Result
`cd backend && DATABASE_URL=sqlite:///<temp>.db ../../python-academy/backend/.venv/bin/python -m pytest -q`	**611 passed, 11 skipped** (541 before, plus 70)
Same with `TEST_POSTGRES_URL=<local PostgreSQL 16 from pgserver>`	**622 passed**
`cd frontend && npx tsc --noEmit && npm run build`	OK

Command	Result
`cd frontend && CI=1 npx playwright test`	**200 passed** (173 before, plus 27)
axe-core 4.10.3 (WCAG 2.0, 2.1, 2.2 A and AA, best practices), in a scratch run: each of the 8 recorded results at 1280 px and 375 px	**0 violations; no sideways scrolling**

What the tests cover

S05 asks for	Where
Correct and wrong functions	Real Pyodide in `function-checks.spec.ts`; the judge in `test_function_checks.py`; the page, from recorded real answers
A missing function, a wrong name, a wrong parameter count	All three levels. Defaults, `*args`, `lambda` and a callable object pass
An exception, with the friendly explanation and the line	The worker reports the line; the page shows "Python does not know missing" and "Go to line 2"
An endless loop	Stopped at the limit in the worker, and through the page at 3 seconds
Hidden values never sent	A hidden call is refused at load. A hidden ordinary case beside calls keeps its expected output off the plan and out of the result. A case forced to hidden by hand still does not leak its value
Units 1 and 2 unchanged	`unit1-unchanged.spec.ts` and every old fixture are unchanged and pass. Every real lesson's plan keeps its exact keys. An ordinary report keeps its exact keys
The equality rule	26 pairs, including `True`/`1`, `4`/`4.0`, `0.1 + 0.2` with a tolerance, nested values and huge numbers
Content validation	17 ways of writing a bad case, each stopping the load

Unit 3 is not on `main`. Its snapshots are on #32's branch, and are not exercised here.

Mocks: the page tests answer API calls with recorded real answers: real Pyodide reports, answered by the real API. `test_the_recorded_answers_are_the_real_apis` proves they still match. Python, the worker and the page are real.

Deliberate-break controls (each undone; files matched saved copies byte for byte afterwards)

What I broke	Tests that failed
B1. Validation allows a hidden call	**1**: the refusal test

What I broke	Tests that failed
B2. The plan sends a hidden case's expected return value	**1**: the forced-hidden leak test
B3. The judge names the expected value on a hidden case	**1**: the same leak test
B4. Tolerance is ignored	**7**: three equality pairs, a correct solution, the forced-hidden test and the recordings
B5. `True` counts as `1`	**3**: the equality pairs
B6. A return value is never compared	**6**: wrong-value tests and the recordings
B7. Output from before the call counts	**2** Playwright
B8. The worker does not check parameters first	**2** Playwright, including the page
B9. The runner stops passing the call on	**26** Playwright

The two unit 2 recording tests, fixed afterwards (section 5), also failed in the backend runs of B1 to B6. They are left out of the counts above. Breaks B1 to B3 being caught by one test each is what is expected of a single guarded line, but it is thin; the full lesson stage would add more.

7. Known limitations

- **No real lesson uses this,** so there is no live check for it yet.
- **No hidden calls** (question 1). A learner can hard-code the visible answers. Until this is decided, S08 and S09 could only catch that with hidden ordinary cases.
- **It is practice-level evidence,** like every browser check. Someone who edits their browser's reports can claim any value (contract section 5).
- **Sets and objects cannot be compared yet.** Unit 4 does not need it for functions, but unit 11 will.
- `returned` **is capped at 10,000 characters as JSON.** Longer values are `not_comparable`, and the message names the type, not the length.
- **The worker changes and #32's worker fix are in the same function.** A trial merge shows no conflict. It does not matter which merges first.
- **The run log conflicts with #33.** Both add `docs/run-log.md`. When one merges, keep the copy with the most rows.
- **Not tested with a real screen reader,** or on Safari or Firefox.

8. Live checks for you

Nothing to check until a lesson uses it. After merging and deploying, confirm that nothing else moved:

- [ ] **Unit 1:** "Open the story" looks and behaves as before.

- [ ] **Unit 2:** in "Count the treasure", card 5, a wrong answer says "Not yet" and a right one says "Check passed".
- [ ] **An endless loop:** `while True: pass` in a unit 2 challenge stops after about 3 seconds.
- [ ] **No CSP reports** in DevTools → Console.

9. Questions for you

Each recommendation is **recommended, not yet confirmed**.

1. **Hidden function calls, the important one.** The plan asks that hidden arguments never reach the browser, but a function runs in the browser. So today calls must be visible, and a learner could hard-code their answers. *Recommended: allow hidden calls whose arguments reach the browser and whose expected values stay on the server. This matches what contract section 5 already accepts for hidden inputs, since knowing an input does not give the answer. It is a one-rule change with its tests. Stages S08 and S09 should wait for this answer.*
2. **Assumptions 2 and 3:** a tuple equals a list, and `True` does not equal `1`. *Recommended: keep both.*
3. **Sets and objects as return values.** *Recommended: add them when a lesson needs them.*

10. For the docs sync

Where	What
Backlog ENG-06	proposed → in_review (#34, handoff 29)
Stage table	"29. ENG-06 | Function-call checks | in_review (#34, handoff 29)"
Decision log	A chapter from sections 4 to 6 of this handoff
Open questions	Question 1

11. Next concrete action

Answer question 1, then review and merge. It doesn't matter whether #32 merges first.

12. Removing the worktree safely

From the **main** folder, after this PR is merged:

```
cd /Users/DEV/Projects/python-academy
git -C ../python-academy-fn status --short   # expect nothing, or only ??
frontend/AGENTS.md
rm -f ../python-academy-fn/frontend/AGENTS.md   # generated by Next.js, not part of the
project
```

```
git worktree remove ../python-academy-fn     # deletes only that folder; refuses if
anything is uncommitted
git worktree prune
git branch -d eng/function-checks            # refuses if the branch is not merged
```

Don't delete the folder in the Finder or with `rm -rf`; Git would keep a stale record of it (`git worktree prune` fixes that).